



Institut F. Vidal i Barraquer

DAW2A - 0613/M7

Desenvolupament web en entorn servidor

Autor:

Pol Poblet

Desembre, 2025

Índex

1. Introducció.....	3
2. Instal·lació manual.....	4
2.1 Preguntes per reflexionar.....	5
3. Posada en marxa amb Docker.....	6
3.1 Instal·lació i configuració.....	6
3.2 Preguntes per reflexionar.....	6
4. CRUD MVC amb Laravel.....	7
4.1 Creació del model i la base de dades.....	7
4.2 Creació del controlador i les rutes.....	8
4.3 Creació de les vistes Blade.....	8
4.4 Implementació del CRUD.....	9
4.5 Prova del funcionament.....	10
6. Annexos.....	11

1. Introducció

Aquest document conté informació de Laravel tant així com la instal·lació i configuració d'aquest. La tasca es realitza de manera manual, instal·lant totes les eines, i amb Docker, on està tot configurat.

També es responen preguntes formulades respecte a Laravel, Docker i l'espai de treball, identificant les diferències entre aquests dos.

2. Instal·lació manual

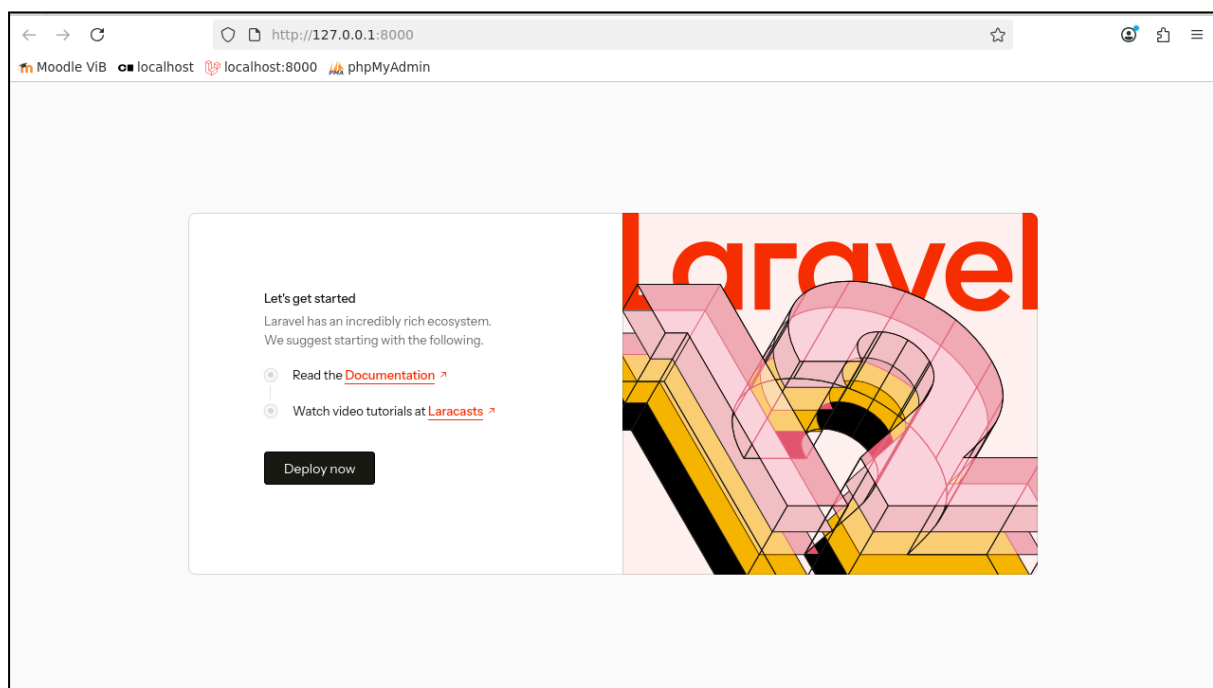
Per realitzar la instal·lació a **Laravel**, hem seguit les dues guies citades als Annexos, escrites per **Composer** i **Kinsta**.



I per resoldre els inconvenients després de crear el projecte, haurem d'executar les següents comandes:

```
sudo apt install sqlite3  
  
cd app-name/  
  
sudo php artisan migrate
```

Una vegada hem instal·lat i configurat tot, entrarem a localhost:8000/, on podrem veure la pantalla d'inici de Laravel.



2.1 Preguntes per reflexionar

1. Quin és l'arxiu principal que executa Laravel? (pista: dins la carpeta public/)

L'arxiu principal és l'`index.php`

2. On estan definides les rutes per defecte d'un projecte Laravel nou?

Estan definides a l'arxiu `routes/web.php`

3. Quin avantatge té utilitzar php artisan serve durant el desenvolupament?

El principal avantatge és poguer veure en temps real la web

4. Què creus que fa realment Composer quan creem un projecte?

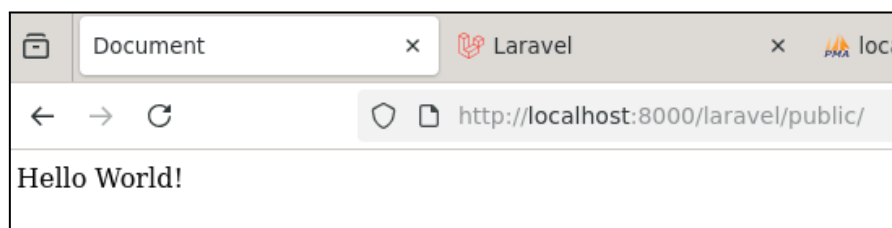
Comprova l'arxiu `package.json` i actualitza o instal·la les dependències afegides.

3. Posada en marxa amb Docker

3.1 Instal·lació i configuració

Primer, descarregarem la carpeta proporcionada pel professorat. La descomprimirem dins la carpeta `srv/` i executarem `make up`. En el meu cas han sorgit un error de la base de dades, i l'he pogut solventar canviant la configuració de la base de dades `mysql` a l'arxiu `.env`. Una vegada hem arreglat el problema anterior, executarem `make art cmd="migrate"`.

Per finalitzar, editarem l'arxiu `resources/views/welcome.blade.php` per poder mostrar un "Hello World!".



En cas que apareixi un error 500, executarem `chmod -R 777 .` sobre la carpeta `/php-Laravel`.

3.2 Preguntes per reflexionar

1. Quina diferència hi ha entre executar Laravel amb `php artisan serve` i fer-ho amb Docker?
Amb docker s'executen tots els serveis necessaris, i estàn agrupats podent executar tots aquests en una sola comanda.
2. Quins avantatges ofereix treballar amb una recepta Docker ja configurada?
No s'ha d'instal·lar tantes eines per poder posar en marxa Laravel
3. Què creus que passa si dos projectes intenten utilitzar el mateix port (per exemple, el 8000)?
Només funcionarà un projecte

4. CRUD MVC amb Laravel

4.1 Creació del model i la base de dades

Crearem un model anomenat **Videojoc** utilitzant la comanda `make art cmd="make:model Videojoc -m"`. També afegirem l'estructura de la taula a `database/migrations/*videojocs_table.php`.

```
*/  
public function up(): void  
{  
    Schema::create('videojocs', function (Blueprint $table) {  
        $table->id();  
        $table->string('nom')->unique();  
        $table->string('plataforma');  
        $table->year('any_estrena');  
        $table->enum('estat', ['Jugant', 'Completat', 'Pendent']);  
        $table->decimal('preu');  
    });  
}
```

Després executarem `make art cmd="migrate"` i indicarem els camps que es poden omplir a `app/Models/Videojoc.php`.

```
Click to create DevDB database connection for Eloquent actions  
class Videojoc extends Model  
{  
    protected $fillable = ['nom', 'plataforma', 'any_estrena', 'estat', 'preu'];  
    //  
}
```

4.2 Creació del controlador i les rutes

Crearem el controlador de **tipus resource** amb la comanda `make art cmd="make:controller VideojocController --resource"`. Definirem les rutes al fitxer `routes/web.php`.

```
// Videojocs
use App\Http\Controllers\VideojocController;

Route::resource('videojocs', VideojocController::class);
```

Amb això, es crearà automàticament les rutes com: `/videojocs`, `/videojocs/create`, etc...

4.3 Creació de les vistes Blade

Crearem la carpeta `resources/views/videojocs` i afegirem diversos arxius a través de la següent comanda:

```
s/views/videojocs# touch index.blade.php create.blade.php edit.blade.php _form.blade.php
```

També crearem la plantilla base, `resource/views/layout.blade.php`.

En les vistes afegirem taules HTML, formularis amb `@csrf`, errors amb `@error` i enllaços per la navegació.

```
@if (session('status'))
    <p style="border:1px solid #ccc; padding:8px;">{{ session('status') }}</p>
@endif

<nav>
    <div>
        
        <ul>
            <li><a href="/videojocs">Home</a></li>
            <li><a href="/videojocs/create">New</a></li>
        </ul>
    </div>
</nav>

<main>
    @yield('content')
</main>
```

4.4 Implementació del CRUD

Completarem el **VideojocController** afegint els següents mètodes:

Llistar tots els videojocs
(index)

```
/**
 * Display a listing of the resource.
 */
public function index()
{
    $videojocs = Videojoc::orderBy('id')->paginate(10);
    $headers = ['ID', 'Nom', 'Plataforma', 'Any Estren', 'Estat', 'Preu'];
    return view('videojocs/index', compact('headers', 'videojocs'));
}
```

Mostrar el formulari d'afegir
(create)

```
/**
 * Show the form for creating a new resource.
 */
public function create()
{
    return view('videojocs.create');
}
```

Desar un videojoc nou
(store)

```
/**
 * Store a newly created resource in storage.
 */
public function store(Request $request)
{
    $validated = $request->validate([
        'nom' => 'required|string|max:100|unique:videojocs,nom',
        'plataforma' => 'required|string|max:100',
        'any_estren' => 'required|integer|min:1980|max:'.date('Y'),
        'estat' => 'required|in:disponible,completat,prestat',
        'preu' => 'required|numeric|min:0',
    ]);

    $videojoc = new Videojoc($validated);
    $videojoc->save();

    return redirect()->route('videojocs.index')->with('status', 'Videojoc creat correctament');
}
```

Mostrar el formulari d'editar
(edit)

```
/**
 * Show the form for editing the specified resource.
 */
public function edit(Videojoc $videojoc)
{
    return view('videojocs.edit', compact('videojoc'));
}
```

Actualitzar un videojoc
(update)

```
/**
 * Update the specified resource in storage.
 */
public function update(Request $request, Videojoc $videojoc)
{
    $validated = $request->validate([
        'nom' => 'required|string|max:100|unique:videojocs,nom, '.$videojoc->id,
        'plataforma' => 'required|string|max:100',
        'any_estren' => 'required|integer|min:1980|max:'.date('Y'),
        'estat' => 'required|in:disponible,completat,prestat',
        'preu' => 'required|numeric|min:0',
    ]);

    $videojoc->update($validated);

    return redirect()->route('videojocs.index')->with('status', 'Videojoc actualitzat correctament');
}
```

Eliminar un videojoc
(destroy)

```
/**
 * Remove the specified resource from storage.
 */
public function destroy(Videojoc $videojoc)
{
    $videojoc->delete();

    return redirect()->route('videojocs.index')->with('status', 'Videojoc eliminat correctament');
}
```

Aplicarem les validacions amb `$request->validate([...])`, això permetrà comprovar si s'han complert els següents punts:

- Els camps obligatoris s'han omplert
- El nom és únic
- El preu i l'any són numèrics

Després de cada operació aplicarem la tècnica **PRG** utilitzant: `return redirect()->route('videojocs.index')->with('status', 'Missatge d'èxit');`.

```
$videojoc->delete();
return redirect()->route('videojocs.index')->with('status', 'Videojoc eliminat correctament');
```

Mostrarem el missatge d'èxit a la vista (`session('status')`).

```
@if (session('status'))
    <p style="border:1px solid #ccc; padding:8px;">{{ session('status') }}</p>
@endif
```

4.5 Prova del funcionament

Abans de realitzar qualsevol acció, afegirem el timestamps al model per no permetre la inserció d'algunes columnes (e.g., update_at).

```
Click to create DevDB database connection for Eloquent actions
class Videojoc extends Model
{
    protected $fillable = ['nom', 'plataforma', 'any_estrena', 'estat', 'preu'];
    public $timestamps = false;
}
```

Ens dirigirem a localhost:8001 i comprovarem que les funcions implementades anteriorment (crear, llistar, editar i eliminar) funcionen.

Afegir

Videojoc creat correctament

Home New

Videojocs +

ID	Nom	Plataforma	Any Estrena	Estat	Preu	Accions
1	Videojoc1	Plataforma1	2000	Jugant	49.99 €	Editar Eliminar
2	crear	pcrear	2025	Jugant	0.00 €	Editar Eliminar

Editar

Videojoc actualitzat correctament

Home New

Videojocs +

ID	Nom	Plataforma	Any Estrena	Estat	Preu	Accions
1	Videojoc1	Plataforma1	2000	Jugant	49.99 €	Editar Eliminar
2	editar	peditar	2025	Pendent	0.00 €	Editar Eliminar

Eliminar

Videojoc eliminat correctament

Home New

Videojocs +

ID	Nom	Plataforma	Any Estrena	Estat	Preu	Accions
1	Videojoc1	Plataforma1	2000	Jugant	49.99 €	Editar Eliminar

També s'ha comprovat que els missatges d'error apareixen.

Afegir videojoc

Nom

The nom field is required.

Plataforma

The plataforma field is required.

Any d'estrena

Estat

Preu

[Guardar](#) [Tornar](#)

6. Annexos

Documents d'interès

Composer (19 Novembre, 2025). *Download Composer*. [\[enllaç\]](#)

Kinsta (15 Octubre, 2025). *How To Install Laravel on Windows, macOS, and Linux*. [\[enllaç\]](#)

GitHub

Pol Poblet Pallisé. *Repositori de GitHub*. [\[enllaç\]](#)